

Research Briefing on Listing the Cache Storage Service and HTML Transformation Service on AWS Marketplace

Executive briefing

You have two technically mature, API-first services that you want to commercialise and learn from by listing on AWS Marketplace:

- A **cache/storage service** that abstracts structured file/object persistence (IDs, hashes, sidecar data, “filesystem as database”), sitting atop a “memory filesystem” abstraction that can target S3/cloud object storage, local disk, in-memory, zip, SQLite, etc.
- An **HTML transformation service** (FastAPI) that deconstructs HTML into components, applies transformation/classification logic, then reconstructs output (useful for filtering, content manipulation, and preprocessing workflows).

On AWS Marketplace, there is no “arbitrary product type”; you must choose an approved **delivery/category**. The Marketplace buyer guide and seller docs explicitly frame software product delivery as: **AMI** (single AMI or CloudFormation deployed topology), **container**, **SaaS**, plus other categories (professional services, data products, etc.). ¹

For your two services, the most relevant options are:

- **SaaS listing** (you host the service; customers subscribe and then access it in *your* AWS environment, or via a VPC endpoint connection you create). ²
- **Container product listing** (buyers deploy your containers into *their* ECS/EKS/Fargate or even other Kubernetes environments; AWS Marketplace delivers images and templates, and can support paid usage or contract pricing). ³
- **AMI product listing** (buyers deploy your software via EC2 instances, optionally via CloudFormation templates to create a full “stack” topology). ⁴

A key decision is whether the **buyer runs the service in their account** (AMI/container) versus **you run it** (SaaS). This affects IP distribution, onboarding complexity, metering integration, and customer data residency expectations. ⁵

Marketplace delivery models that map well to API services

What AWS Marketplace optimises for

AWS Marketplace is structured around (a) packaging/delivery, (b) procurement and billing via AWS, and (c) security/compliance controls like scanning and policy requirements. For example, AWS explicitly frames that products can be delivered as AMIs (including CloudFormation delivery), SaaS offerings, and more, and that pricing options include free trial, hourly, monthly, annual, multi-year and BYOL—with AWS handling billing and surfacing charges on the customer’s AWS bill. ⁶

For API products like yours, there are two common commercial “shapes” in Marketplace:

- **Self-hosted API (buyer deploys):** container-based product or AMI-based product, typically with deployment templates and clear usage instructions; billing can be hourly/monthly/contract/custom-metered, or BYOL. ⁷
- **Managed API (vendor hosts):** SaaS listing where the buyer subscribes, is redirected to your fulfilment/registration URL, and you validate entitlements/meter usage using AWS Marketplace APIs depending on pricing model. ⁸

A practical decision matrix

Dimension	SaaS listing (you host)	Container product (buyer hosts)	AMI product (buyer hosts)
Who operates the runtime	You (provider)	Buyer	Buyer
Typical fit for your services	External “managed API” offering; fastest adoption for dev teams who just want an endpoint	Strong fit for FastAPI services; natural for “run it in my VPC / cluster” customers	Good for customers who prefer EC2 appliances or need simple VM deployment
Marketplace integration complexity	Highest (customer onboarding, entitlement checks, metering, subscription events) ⁹	Medium (packaging + Marketplace container policies + optional metering/licensing) ¹⁰	Medium (AMI build + AMI policies + scanning + optional metering/licensing; can add CloudFormation templates) ¹¹
Customer data residency	Data likely flows to your AWS account unless you design otherwise	Data stays in buyer’s account by default	Data stays in buyer’s account by default
“Serverless Lambda listing” viability (2026)	Use Lambda internally if you want, but SaaS listing is how it’s sold	Not relevant	Partially relevant via CloudFormation add-ons, but new SAR-based publishing is restricted (see below) ¹²

Important 2026 note on “Lambda as a Marketplace artefact”: AWS Marketplace documentation states it **no longer supports publishing new products** with CloudFormation templates that deploy resources from **AWS Serverless Application Repository (SAR)**, though existing ones can continue until a future date. ¹³

This doesn’t stop you from running your SaaS backend on Lambda, but it does affect older patterns of “sell a serverless app by referencing SAR in CloudFormation.” ¹⁴

Recommended listing strategies for your two services

Cache storage service

Strongest first listing: container product (self-hosted API)

Your cache service is a foundational infrastructure component. Many AWS customers will prefer it running *inside their own account* (data stays in their VPC/S3, IAM under their control). A **container-based Marketplace product** is well-aligned because container products are explicitly designed as “a set of container images and deployment templates,” deployable to ECS/EKS/Fargate and even other Kubernetes environments. ¹⁵

Key implications you must design for:

- **All container images must be pushed to AWS Marketplace-managed ECR repositories** (not Docker Hub / external registries). ¹⁶
- Container images must be **Linux-based**, and must not contain known vulnerabilities/EoL packages. ¹⁷
- Your service must **not request AWS credentials**; when accessing AWS services (e.g., S3), you must rely on IAM roles (for EKS, IAM roles for service accounts are explicitly referenced as an approved approach). ¹⁸

Suggested Marketplace “fulfilment options” (delivery options) for the cache service:

- **ECS/Fargate deployment template** (quick path for teams not using Kubernetes).
- **Helm chart** for EKS (but avoid relying on Marketplace Quick Launch for EKS Helm—see the deprecation note later). ¹⁹
- Optional: “local disk cache” vs “S3-backed cache” via configuration, but be careful: container product policies around collecting sensitive configuration mean you should pass secret *names* (Kubernetes secrets) rather than embedding credentials in config schemas. ¹⁶

Alternate listing: AMI product with CloudFormation templates (EC2 appliance)

If you expect customers who want a VM “appliance” style deployment, you can provide an **AMI-based** listing. AWS explicitly supports AMI products delivered as a **single AMI** or delivered through **CloudFormation templates** that install a system of resources as a unit. ²⁰

If you go the CloudFormation route, AWS Marketplace’s CloudFormation support is notable:

- You can add CloudFormation templates to an AMI product to deploy dependencies without manual set-up; templates can define clustered/distributed architectures. ²¹
- Single-AMI solutions can include **up to three CloudFormation templates**, and templates can deliver a single AMI plus associated config files and **Lambda functions** (useful for automation/bootstrap). ²²
- AWS has template prerequisites such as “launch successfully in all enabled regions,” avoid AZ-specific assumptions, and (even for single-node) recommends using an Auto Scaling group. ²³

Managed offering: SaaS cache API (later / optional)

If you want a “managed cache API” (especially for developers who want a hosted endpoint quickly), SaaS can work—but it adds onboarding and metering integration overhead. SaaS products must comply with Marketplace SaaS rules on billing, customer onboarding and “web console” availability. ²⁴

HTML transformation service

Your HTML transformation service can plausibly be sold as either:

- A **self-hosted API** (container/AMI) for customers whose HTML content is sensitive, or they need VPC-contained processing.
- A **managed SaaS API** when customers want convenience and don’t want to operate anything.

Container listing fit

As with the cache service, the HTML service is a FastAPI workload that packages cleanly into containers. Container products allow up to four delivery options, each with deployment templates and usage instructions. ¹⁹

Given that HTML processing often touches sensitive content, a buyer-hosted container deployment can be easier to sell into regulated environments.

SaaS listing fit (if you build the minimum required console)

For SaaS, AWS requires a specific onboarding experience:

- You provide a **product registration/fulfilment URL**; after subscribing, AWS redirects buyers there. ²⁵
- The registration landing page must accept `x-amzn-marketplace-token` via POST, then call **ResolveCustomer** to obtain customer identifiers; for contract models, you also check entitlements. ²⁶
- AWS Marketplace SaaS guidelines require (among other things) that SaaS is billed entirely through Marketplace dimensions, you cannot collect payment details, the registration page includes at least an email field, and customers should gain access to a **web console** (even if it’s primarily API-focused, you generally need some console for account/subscription visibility). ²⁷

This suggests a pragmatic SaaS approach for the HTML service is to implement a minimal “developer console” that issues API keys, shows usage/subscription status, and provides an endpoint base URL—satisfying the “web console” expectation while keeping engineering minimal. ²⁸

Step-by-step paths to publish

Seller onboarding and product lifecycle

Regardless of delivery type, publishing on AWS Marketplace starts with **seller registration**. AWS docs describe seller registration as a prerequisite to listing products, with different requirements depending on free vs paid offerings. For free products, AWS notes you still need production-ready software, a defined support process, and a way to keep software updated and free of vulnerabilities. ²⁹

For selling generally, AWS describes registration steps including entering banking and tax information.

³⁰

After you create a product, Marketplace uses well-defined states such as **Staging**, **Limited**, **Public**, and **Restricted**, with Limited being the allowlisted/test state before going public. ³¹

Path for a container product (recommended “first listing” approach)

AWS’s “getting started with container products” guide describes a workflow:

- Create product ID and product code (a public key pairing is also created). ³²
- Create an initial listing (for paid products, AWS notes initial \$0.01 pricing for testing; you set final pricing at public launch). ³³
- Add versions, and for paid products integrate metering or contract pricing. ³⁴
- Update visibility from Limited to Public; this involves Seller Operations review. ³¹

Operationally, AWS scans container images when you add a new version and will flag critical vulnerabilities; they recommend you run your own scanning to avoid delays. ³⁵

Path for an AMI product (including CloudFormation topology)

An AMI product is delivered as a custom AMI; buyers launch EC2 instances and are billed according to the pricing/metering you set. ³⁶

AWS allows AMI delivery via CloudFormation templates to deploy multi-instance topologies. ²⁰

Two practical implications for your architecture:

- You can bundle a “distributed” setup (e.g., Auto Scaling group + load balancer + optional extras) into CloudFormation templates as part of the Marketplace listing, giving buyers a near “one click” deployment. AWS explicitly positions this as a way to deploy dependencies without manual configuration. ³⁷
- Marketplace policy constraints for AMIs are strict: AMIs must pass AWS Marketplace scanning with no known vulnerabilities/malware; must use supported OS/software; must not contain hardcoded secrets; must not request AWS credentials (customers should assign minimally privileged IAM roles instead). ³⁸

Path for a SaaS product

AWS’s SaaS creation guide lays out broad steps: create the SaaS product, integrate based on offer type (subscription/contract/contract with pay-as-you-go), test integration, and submit for launch. ³⁹

It also lists the assets you must collect, including a product logo URL, EULA URL, and crucially the **product registration URL** where buyers are redirected after subscribing. ³⁹

From an engineering standpoint, the most load-bearing requirement is the onboarding flow:

- The buyer lands on your registration URL via POST with `x-amzn-marketplace-token`. ⁴⁰
- Your app calls **ResolveCustomer** to translate that token into `CustomerIdentifier`, `CustomerAWSAccountId`, and `ProductCode`. ⁴¹
- For **contract** models you check entitlements via `GetEntitlements`; for subscription models you focus on metering usage; SaaS guidelines explicitly map which AWS Marketplace APIs must be called depending on pricing type. ⁴²
- You then persist customer identifiers and grant access; AWS expects you to handle subscription/entitlement changes (AWS notes SNS is being replaced by EventBridge for SaaS subscription events, with new listings transitioning). ⁴³

Security, compliance, and deployment implications you must bake in

Secrets and credentials

For both AMI and container Marketplace products, AWS policies strongly push you away from credential-based set-ups:

- AMI products must not contain hardcoded secrets; must not request AWS credentials; if AWS service access is needed, the instance should be assigned a minimally privileged IAM role. ⁴⁴
- Container products similarly must not request AWS credentials; IAM roles/service account approaches are specified for AWS service access. ¹⁶

This intersects directly with your current approach of passing secrets via CI into environment variables. For Marketplace products, plan to shift toward IAM roles (instance profile, task role, IRSA) and customer-managed secrets mechanisms, rather than shipping any embedded keys. ⁴⁵

Vulnerability scanning and update cadence

AWS Marketplace enforces scanning and ongoing compliance:

- AMIs must pass Marketplace scanning checks (no known vulnerabilities/malware); supported OS/software only. ³⁸
- Container images are scanned layer-by-layer on new version submission; AWS flags critical vulnerabilities with remotely exploitable risk vectors. ³⁵
- AWS also highlights that products can become temporarily unavailable to new subscribers if they fall out of compliance (e.g., updated security requirements). ⁴⁶

This means your Marketplace plan should include an explicit “security patch release” process with automated rebuilds and scans.

Quick Launch and upcoming deprecations

AWS Marketplace supports **Quick Launch** as a guided CloudFormation-based deployment option for SaaS and container products. ⁴⁷

However, AWS’s container products documentation states that on **March 1, 2026** AWS Marketplace will discontinue **Quick Launch for Helm chart deployments on Amazon EKS**; existing deployments continue, and you can still deploy via standard Helm commands or container images on ECS. ⁴⁸

Given today’s date (January 29, 2026), you should treat “Helm Quick Launch” as a near-term sunset feature and prefer:

- Helm charts deployable via normal Helm flows (still valid), and/or
- ECS/Fargate templates for a simple managed path. ¹⁹

Private connectivity options for SaaS

If you list a SaaS product, you can offer access over the internet via your own website, or you can configure your SaaS as a VPC endpoint service using **AWS PrivateLink**, letting customers access it across the AWS virtual network. ⁴⁹

AWS Marketplace and Amazon VPC documentation describe discovering and provisioning SaaS products

powered by PrivateLink, and the VPC guide describes creating endpoints to connect to Marketplace services. ⁵⁰

For enterprise buyers, “PrivateLink-powered SaaS” can be a major commercial lever (security posture, network control). Even if you start with public SaaS endpoints, it’s worth structuring your hosting architecture so that PrivateLink can be layered in later (typically NLB + endpoint service). ⁵¹

Competitive landscape and positioning signals from existing Marketplace listings

Caching and “proxy/filtering” adjacent products are already present

AWS Marketplace already contains mature caching products such as **Varnish Pro / Varnish Enterprise** listings. ⁵²

It also has multiple **Redis**-oriented listings and in-memory database products, reflecting that “cache” as a category is crowded. ⁵³

Separately, Marketplace also contains web filtering proxy appliances based on Squid (e.g., “Web Safety ... web filtering proxy appliance”), showing that “proxy/filtering” is a legitimised Marketplace category. ⁵⁴

Implication for your cache service positioning: avoid competing head-on with Redis/Varnish on “generic caching”. Instead, position around the differentiators you described:

- **Content-addressable + structured object layouts** (IDs, hashes, metadata/sidecars) as a reusable substrate for “filesystem as database”.
- **LLM workflow storage** primitives (load/extract/transform/save) and reproducible pipeline artefacts.
- **Pluggable backends** (S3, disk, memory, zip, SQLite) as a portability story, with S3 as the cloud-grade store.

This is closer to “developer productivity for data/LLM pipelines” than “generic cache”.

HTML transformation “API products” are common, but your angle is different

AWS Marketplace contains multiple document/HTML transformation APIs (for example tools that convert HTML/web pages to PDF, or parse documents into HTML). ⁵⁵

These listings indicate that buyers are comfortable purchasing “transformation APIs” via Marketplace, both self-hosted and service-style.

Your HTML service’s differentiator is semantic-aware manipulation/sanitisation and “deconstruct/reconstruct HTML” for filtering and transformation, which can be positioned as:

- A content safety / compliance preprocessor for AI and web pipelines.
- A deterministic HTML-to-HTML transformation layer for secure presentation and policy enforcement.

Pricing and packaging options that fit your services

What pricing models are available

AWS Marketplace pricing varies by product type:

- **SaaS** supports subscriptions (pay-as-you-go), contracts, contracts with pay-as-you-go, and free; importantly, once you publish to Limited you **can't change the SaaS pricing model**, and AWS further notes pricing model change is not supported for SaaS products. ⁵⁶
- **Container products** can be free, BYOL, or paid; AWS notes pricing is global across regions and describes rules for price changes and notifications. ⁵⁷
- **AMI products** can be BYOL, paid hourly, hourly-annual, monthly combinations, and contract pricing; AWS also supports private offers for multi-year/custom duration contracts. ⁵⁸

Across Marketplace, AWS summarises flexible options such as free trial, hourly, monthly, annual, multi-year, BYOL, with billing handled by AWS and shown on the AWS bill. ⁵⁹

Pricing dimensions that map well to your two services

AWS Marketplace metering supports **up to 24 dimensions** in various contexts (custom metering and contracts). For example, AWS Marketplace Metering Service supports usage categories such as users/data/bandwidth/hosts/unit, and custom metering for AMI and containers often uses up to 24 dimensions. ⁶⁰

A practical approach is to define dimensions that mirror how customers think about value:

For the **cache storage service**:

- "GB stored" or "GB-month stored" (data category), optionally with tiers for storage class or retention policy.
- "Requests" or "operations" (e.g., read/write/list) if your workloads are request-heavy.
- "Bandwidth" (egress) only if you are actually a data-transfer-heavy service and want to price for it.

For the **HTML transformation service**:

- "Pages processed" (requests)
- "MB processed" (data)
- Optional "advanced transforms" dimension (units) if you have expensive semantic/classification calls that can be counted.

If you choose **container custom metering**, AWS's container pricing docs describe custom metered pricing dimensions and the general concept of multiple dimensions. ⁶¹

If you choose **AMI custom metering**, AWS notes you select a usage category and define up to 24 dimensions, metered hourly, and you cannot change the dimension after the AMI is published. ⁶²

“Learning-first” commercial packaging approach

If your goal is to learn quickly while still testing commercial potential, a common sequencing is:

- Ship a **container product** first (fastest iteration loop for API software), possibly starting with a straightforward pricing model (monthly fixed price or per-pod/per-task hourly). ⁶³
- Use **Limited visibility** allowlisting to trial deployments with your own accounts and friendly users; Limited status exists specifically for testing. ⁶⁴
- Add a SaaS listing later, once you have clear demand for “managed endpoint” convenience and you’re willing to implement the fulfilment landing page, entitlements and metering flows. ⁶⁵

Automating Marketplace publishing and updates

You’re building an internal “build service” and already think in primitives. AWS Marketplace provides APIs designed for automation:

- AWS Marketplace **Catalog API** can be used by approved sellers to view/update products programmatically and “integrate with product build or deployment pipelines” to automate updates. ⁶⁶
- AWS docs mention the Catalog API can automate container product actions such as adding new versions, updating repositories, and restricting versions. ⁶⁷

This is highly aligned with your approach: treat Marketplace as another deployment target, with “publish new version” as a CI/CD step.

Submission and review mechanics to plan for

AWS describes that when you submit a product request, AWS Marketplace operations review it; once approved you receive a **limited listing URL** to preview/approve, and approving publishes the product. ⁶⁸

Because review queues can take time in practice, designing for fast resubmission loops (clean artefacts, strong documentation, pre-scanned images) materially improves time-to-public.

Bottom line recommendations

A pragmatic, AWS-aligned route is:

- Start with a **container-based Marketplace product** for the cache service and (optionally) the HTML transformation service, because it matches your FastAPI delivery style and keeps customer data inside the buyer’s AWS account while avoiding SaaS onboarding complexity. ⁶⁹
- Offer an **AMI + CloudFormation** variant later if you see demand from EC2-first customers or customers who want a VM appliance model. ²⁰
- Consider a **SaaS managed API** offering once you’re ready to implement the Marketplace fulfilment flow (token handling, ResolveCustomer, metering/entitlements, subscription events) and a minimal web console that satisfies AWS SaaS guidelines. ⁷⁰
- Avoid planning around “selling a Lambda/SAM artefact via SAR references” for new listings: AWS says new products using CloudFormation templates that deploy SAR resources are no longer supported. ¹³

- If you use Helm, don't bank on Marketplace Quick Launch for EKS Helm: AWS states Quick Launch for Helm chart deployments on EKS will be discontinued on **March 1, 2026**; support standard Helm instructions and/or ECS/Fargate templates. ¹⁹

¹ Product types available in AWS Marketplace

https://docs.aws.amazon.com/marketplace/latest/buyerguide/buyer-product-types.html?utm_source=chatgpt.com

² SaaS product pricing in AWS Marketplace

https://docs.aws.amazon.com/marketplace/latest/userguide/saas-pricing-models.html?utm_source=chatgpt.com

³ ¹⁵ ¹⁹ ⁴⁸ ⁶⁹ Container-based products on AWS Marketplace

https://docs.aws.amazon.com/marketplace/latest/userguide/container-based-products.html?utm_source=chatgpt.com

⁴ ²⁰ ³⁶ AMI-based products in AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/ami-products.html>

⁵ ⁶ ⁵⁹ Pricing overview - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/pricing-overview.html>

⁷ ¹⁶ ¹⁷ ¹⁸ ⁴⁶ Container-based product requirements for AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/container-product-policies.html>

⁸ ²⁵ ³⁹ ⁶⁵ Creating a SaaS product in AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/saas-create-product.html>

⁹ ⁴³ Integrating your SaaS contract product with AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/saas-integrate-contract.html>

¹⁰ ³¹ ³² ³³ ³⁴ ³⁵ ⁶⁴ Getting started with container products - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/container-product-getting-started.html>

¹¹ ³⁸ ⁴⁴ ⁴⁵ AMI-based product requirements for AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/product-and-ami-policies.html>

¹² ¹³ ¹⁴ Adding serverless application components - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/cloudformation-serverless-application.html>

²¹ ²² ²³ ³⁷ Add CloudFormation templates to your product - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/cloudformation.html>

²⁴ ²⁷ ²⁸ ⁷⁰ SaaS product guidelines for AWS Marketplace

https://docs.aws.amazon.com/marketplace/latest/userguide/saas-guidelines.html?utm_source=chatgpt.com

²⁶ ⁴⁰ Onboarding customers to your SaaS product through ...

https://docs.aws.amazon.com/marketplace/latest/userguide/saas-product-customer-setup.html?utm_source=chatgpt.com

²⁹ Getting started as an AWS Marketplace seller

https://docs.aws.amazon.com/marketplace/latest/userguide/user-guide-for-sellers.html?utm_source=chatgpt.com

³⁰ ⁶⁸ Selling on AWS Marketplace

https://docs.aws.amazon.com/marketplace/latest/userguide/using-aws-marketplace-as-a-provider.html?utm_source=chatgpt.com

⁴¹ ResolveCustomer - AWS Marketplace

https://docs.aws.amazon.com/marketplace/latest/APIReference/API_marketplace-metering_ResolveCustomer.html?utm_source=chatgpt.com

⁴² SaaS product guidelines for AWS Marketplace - AWS Marketplace

<https://docs.aws.amazon.com/marketplace/latest/userguide/saas-guidelines.html>

- 47 Using Quick Launch with SaaS and container products - AWS Marketplace
<https://docs.aws.amazon.com/marketplace/latest/buyerguide/quick-launch.html>
- 49 Planning your SaaS product - AWS Marketplace
https://docs.aws.amazon.com/marketplace/latest/userguide/saas-prepare.html?utm_source=chatgpt.com
- 50 Access SaaS products through AWS PrivateLink
https://docs.aws.amazon.com/vpc/latest/privatelink/privatelink-access-saas.html?utm_source=chatgpt.com
- 51 Delivering your products through a Amazon VPC using ...
https://docs.aws.amazon.com/marketplace/latest/userguide/privatelink.html?utm_source=chatgpt.com
- 52 Varnish Pro - AWS Marketplace
https://aws.amazon.com/marketplace/pp/prodview-lkmlzlddsck2?utm_source=chatgpt.com
- 53 AWS Marketplace: Redis
https://aws.amazon.com/marketplace/seller-profile?id=749c31c1-ca53-4f37-b51d-03ee030ca20f&utm_source=chatgpt.com
- 54 AWS Marketplace: Web Safety Proxy PAYG - Amazon.com
https://aws.amazon.com/marketplace/pp/prodview-ixvzbzugrltcqq?utm_source=chatgpt.com
- 55 AWS Marketplace: ConvertAPI
https://aws.amazon.com/marketplace/pp/prodview-7uxio6wulkjp6?utm_source=chatgpt.com
- 56 SaaS product pricing in AWS Marketplace - AWS Marketplace
<https://docs.aws.amazon.com/marketplace/latest/userguide/saas-pricing-models.html>
- 57 61 63 Container products pricing for AWS Marketplace - AWS Marketplace
<https://docs.aws.amazon.com/marketplace/latest/userguide/pricing-container-products.html>
- 58 AMI product pricing for AWS Marketplace - AWS Marketplace
<https://docs.aws.amazon.com/marketplace/latest/userguide/pricing-ami-products.html>
- 60 Configuring custom metering for AMI products with ...
https://docs.aws.amazon.com/marketplace/latest/userguide/custom-metering-with-mp-metering-service.html?utm_source=chatgpt.com
- 62 Custom metering pricing for AMI products with AWS ...
https://docs.aws.amazon.com/marketplace/latest/userguide/custom-metering-pricing-ami-products.html?utm_source=chatgpt.com
- 66 Using the AWS Marketplace Catalog API
https://docs.aws.amazon.com/marketplace/latest/APIReference/catalog-apis.html?utm_source=chatgpt.com
- 67 Work with container-based products using the ...
https://docs.aws.amazon.com/marketplace/latest/APIReference/work-with-container-products.html?utm_source=chatgpt.com